Basic Elements

1. Processor
2. Main Memory
3. I/O Module
4. System bus

Processor

Control the operation of the computer and performs its data processing functions

Program Counter (PC)

It holds address of next instruction to fetch from memory.

Instruction Register (IR)

Temporarily holds fetched instruction when its being read by the CPU.

Memory Address Register (MAR)

Specifies the address in memory to be read from MBR (Memory Buffer Register)

Memory Buffer Register (MBR)

It contains words to be stored in memory or it can use to receive word from memory

I/O module

move data blw computer and its external environment.

System Bus

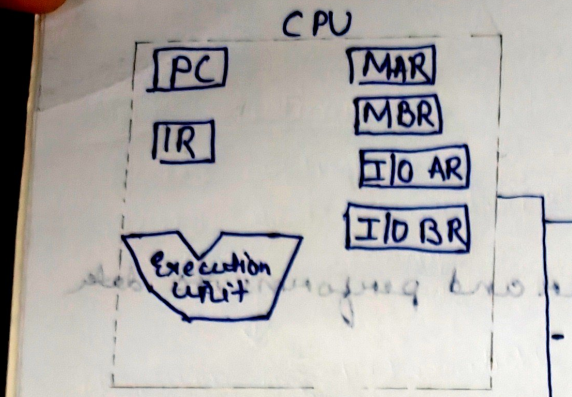
Provides for common among processor, I/O module & main memory.

I/O AR

Input / Output Address Register - specifies particular I/O devices

I/O BR

Input / Output Buffer Register - Exchange of data b/w I/O module & CPU



Main Memory

Instruction

"

"

"

Data

"

"

I/O module

Buffer

Execution of microprocessor

4 bit Mprocessor	8bit	16 bit	32 bit
Intel 4004	intel 8008	intel 8086	intel 80386
Intel 4040	intel 8080	intel 8088	intel 80486
	intel 8085	intel 80186	intel pentium
		80188	intel pentium pro
		intel 80286	

64 bit

Intel core 2

Intel core i7

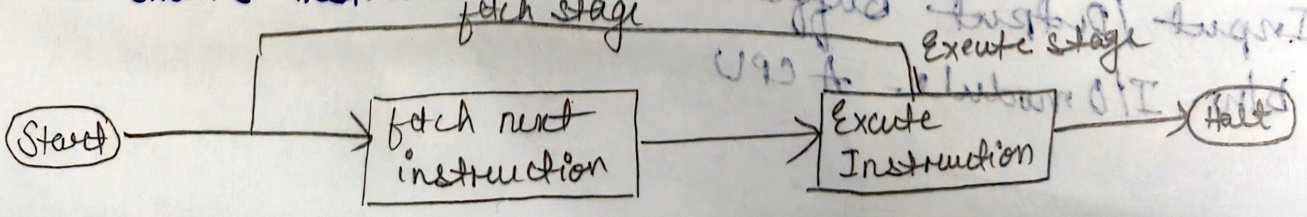
Intel core i5

Intel core i3

Difference bet^w MP i3 i5 i7

Instruction Execution

1. Fetch instruction from main memory
2. Execute instruction



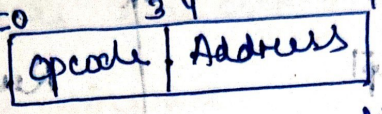
A program to be executed by a processor consist a set of instruction stored in memory. Instruction processing consist of 2 steps :—

- The processor reads from memory
- Execute each instruction
- The fetched instruction is loaded into the IR.
- The processor interrupt the instruction and perform the required action includes processor memory, data processing, processor I/O and control

Instruction Format 1. opcode 2. Address

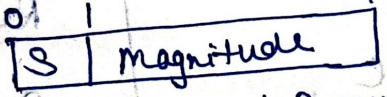
AC – accumulator

a) **Instruction Format**



1st 4 bit instructions
Remaining 12 Address

b) **Integer Format**



1. Program Counter (PC) = Address of Instruction
2. Instruction Register (IR) – Store the instruction Instruction being executed
3. Accumulator (AC) = Temporarily stores Temporary Storage (C) Internal CPU registers

0001 = Loads AC from memory
0010 = " " to "
0101 = Add to " from "

Interrupt

1. Programs
2. Timers
3. I/O
4. Hardware failure

Imp Hardware

Device controller or other systems hardware issue an interrupt

↓

Processor finishes execution of current instruction

↓

Processor signals acknowledgement of interrupt

↓

Processor pushes PSW and PC onto control stack

↓

Processor loads new PC value based on interrupt

Software

Save remainder of processor state information

↓

Process Interrupt

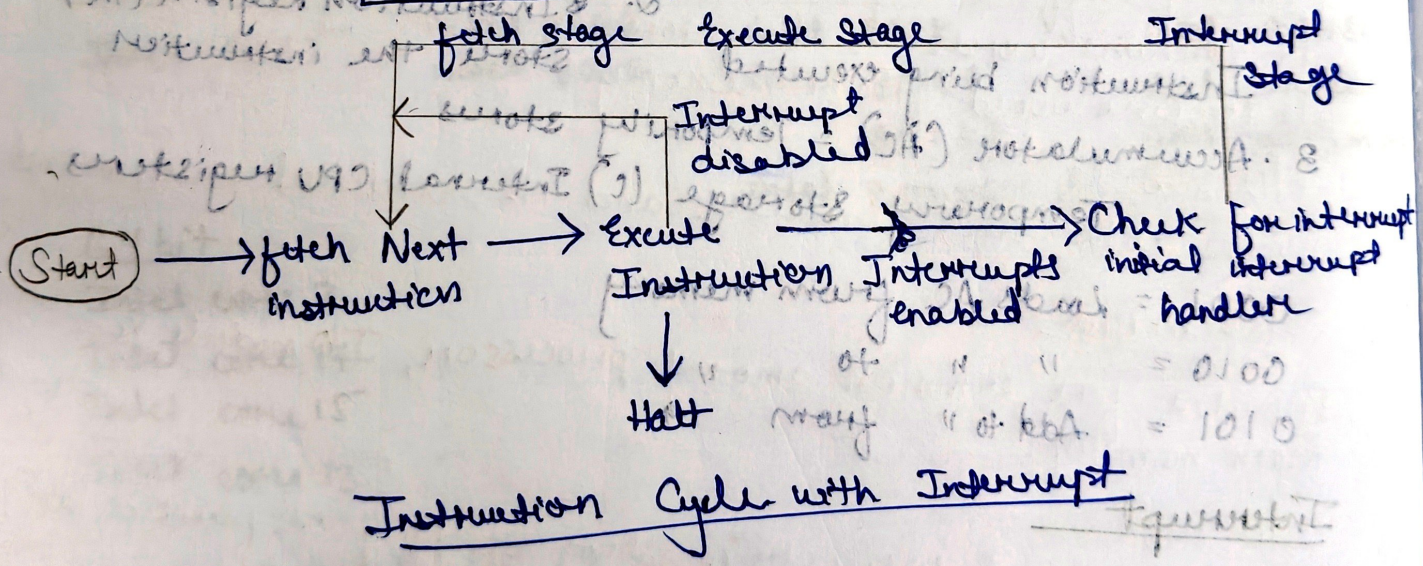
↓

Restore Process' state information

↓

Restore old PSW and PC

Simple Interrupt Processing



Instruction Cycle with Interrupt

Interrupts

Without interrupt

CPU gives I/O command
↓
waits for slow I/O devices
↓
CPU remains idle

With interrupt

CPU gives I/O command
↓
Continues executing other instructions
↓
I/O device finishes
↓
Send interrupt
↓
CPU services it

Interrupts

An interrupt is a signal that causes the CPU to temporarily suspend current executing programs and transfer control to an interrupt service routine or interrupt handler

Need of Interrupt

- The main purpose of interrupt is to improve CPU utilization.

Classes of Interrupt

(i) Program Interrupt – It is generated by program execution, divides by zero, overflow and illegal instruction

(ii) Timer Interrupt – It is generated by processor timer
Examples – Operating system scheduling and time sharing

(iii) IO Interrupt – It is generated by IO controller or devices example – Keyword input, disc completion

(iv) Hardware failure interrupt : generated by hardware failure eg: power failure, memory parity error

Interrupts

Without interrupt

CPU gives I/O command
↓
waits for slow I/O devices
↓
CPU remains idle

With interrupt

CPU gives I/O command
↓
Continues executing other instructions
↓
I/O device finishes
↓
Send interrupt
↓
CPU services it

Interrupt

An interrupt is a signal that causes the CPU to temporarily suspend current executing program and transfer control to an interrupt service routine or interrupt handler

Need of Interrupt

- The main purpose of interrupt is to improve CPU utilization.

Classes of Interrupt

(i) Program Interrupt – It is generated by program execution, divides by zero, overflow and illegal instruction

(ii) Timer Interrupt – It is generated by processor timer
Examples – Operating system scheduling and time sharing

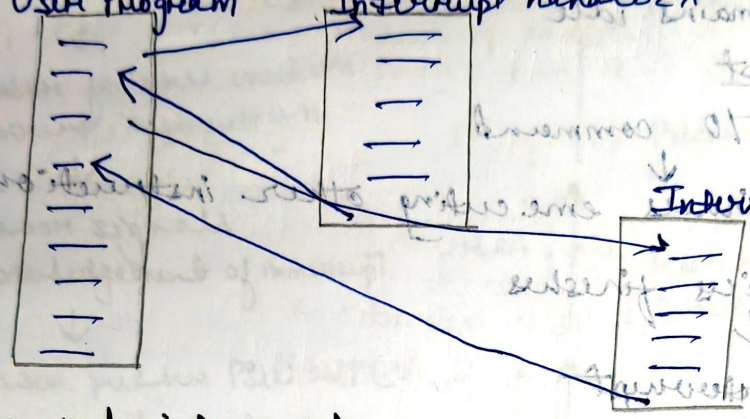
(iii) IO Interrupt – It is generated by IO controller or devices example – Keyword input, disc completion

(iv) Hardware failure interrupt : generated by hardware failure eg: power failure, memory parity error

Multiple Interrupt

1) Sequencial Interrupt Processing

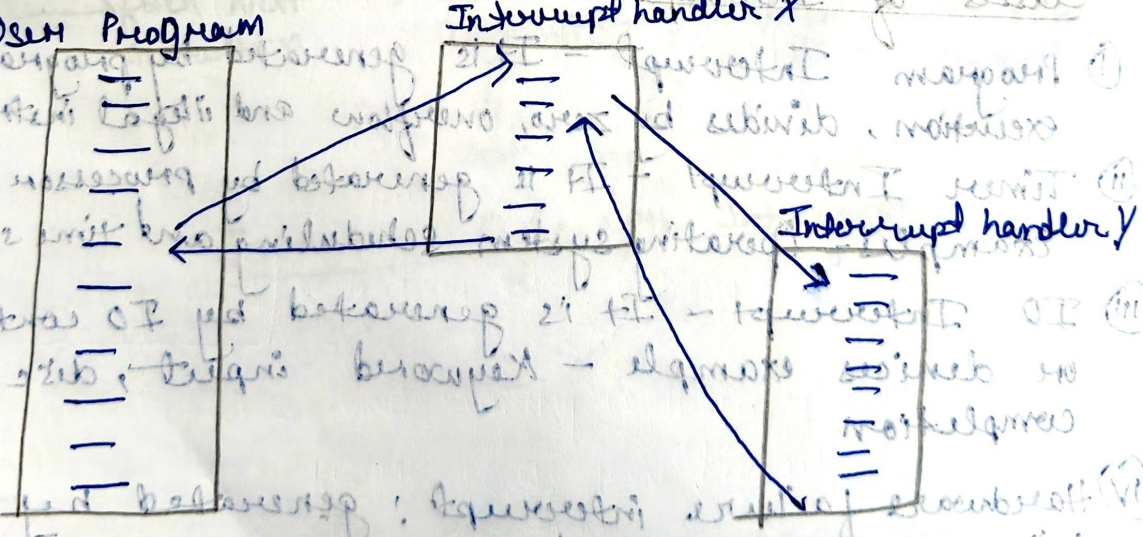
If Interrupt Y occurs whiles ISR 'X' is running ,'y' remains pending and is handled later



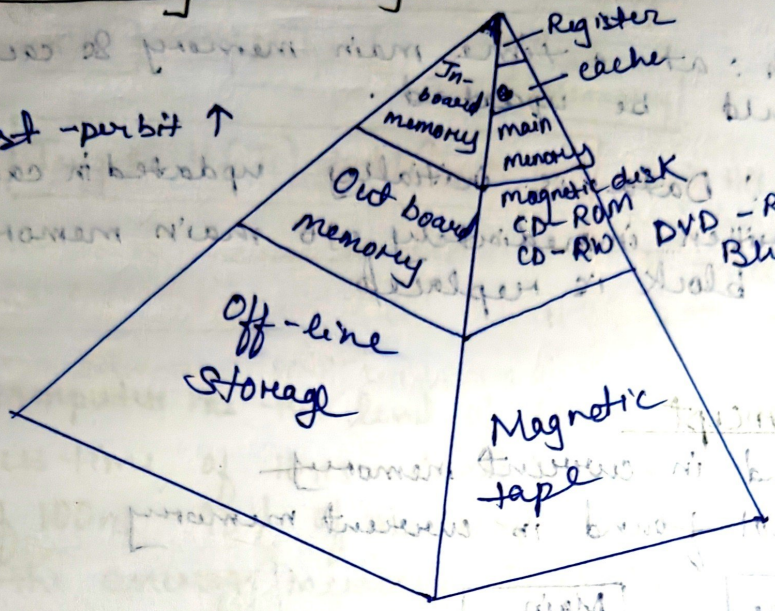
Nested interrupt processing

In nested interrupt processing a higher Priority interrupt can interrupt a currently executing lower priority ISR

Printer priority is = 4
disk " = 4
communication " = 5



The Memory Hierarchy



Top level = faster, smaller, Expensive
Bottom = Slower, larger, Cheap

Cache Memory

diff between cache memory and main memory

Locality of Refrence

Need of Tag

Mapping Function – When a block is brought from main memory to cache memory. Where should this block be placed in the cache memory. This can be done by different maping function – direct mapping, associative mapping, set associative mapping

Write Policy – Suppose the CPU modifies data that is currently in cache memory. Now cache memory and main memory may contain different value. Write policy determine when should be the modified data written back to main memory

~~At a time~~

→ Write through : at a time main memory & cache memory should be updated.

→ Write back : Data is initially updated in cache memory & written immediately to main memory when the block is replaced

Hit and Miss Concept

Hit — Data found in current memory

miss — Data not found in current memory

CPU | Cache | Main Memory

Let 90% is present in cache out of 100

$$\text{Hit rate} = \frac{90}{100} = 0.90$$

$$\text{miss rate} = \frac{10}{100} = 0.10$$

Average Access Time

Q The computer has level 1 cache with an access time of 10 ns The processor accesses the cache memory and 90% of the required data found in cache memory

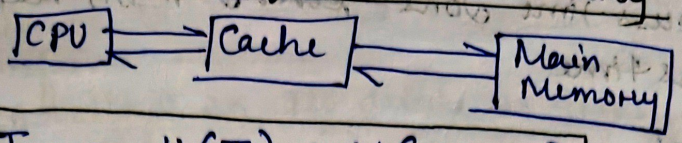
Ans $T_1 = 10ns$ Hit rate = 90% = 0.90

miss = 0.10

$T_{avg} = T_1$ # If only level 1 memory is present

If only one level memory is there then Hit ratio doesn't change the average access time because there is no lower memory level is present.

Average Access time for 2 level Memory



$$T_{avg} = H(T_1) + M(T_1 + T_2)$$

H → Hit rate
M → Miss "
T_1 – access time of level 1
T_2 – " " " level 2

Q) A computer has two level of ~~cache~~ memory. Level 1 has a access time of 10 ns. Level 2 main memory has a access time of 100 ns. 90% of memory access are found in level 1. Find the average memory access time

Soln $T_1 = 10ns$

$T_2 = 100ns$

$H = 0.90 \quad M = 0.10$

$\frac{90}{100} \times 10$

$\frac{90}{100} \times 10$

$\frac{10}{100} \times 110$

$T_{avg} = 0.90 \times 10 + 0.10 \times (110)$

$= 9 + 11 = 20ns$

Average Access time for 3 level memory

$$T_{avg} = H_1(T_1) + M_1H_2(T_1 + T_2) + M_1M_2(T_1 + T_2 + T_3)$$

Q) A computer has 3 level of memory. Level 1 cache memory has an access time of 10 ns. Level 2 ~~has a~~ cache memory has an access time of 50 ns. Level 3 has a cache memory has 100 ns. 90% of memory access is found in Level 1. 80% are found in level 2. Find the average memory access time

Ans $T_1 = 10ns$ $H_1 = 0.90$ $H_2 = 0.80$

$T_2 = 50ns$ $M_1 = 1 - H_1 = 0.10$ $M_2 = 1 - H_2 = 0.20$

$T_3 = 100ns$

$T_{avg} = 0.90(10) + 0.10 \times 0.80(60ns) + 0.10 \times 0.20(160)$

$=$

Q For level 1 memory access time 20ns. ~~Level 2 =~~ Hit rate is 75%. find the average access time

Ans $T_{avg} = T_1 = 20ns$

Q2) A computer as 2 level of memory level 1 access time is 10ns level 2 acess time is 80 ans. 90% of memory access time is found in level 1. Then find the average access time

$T_{avg} = H(T_1) + M(T_1 + T_2)$

$= 0.90 \times 10 + 0.10 \times 90 = 18ns$

Q3) Level 1 acess time 5ns, Level 2 = 20ns, Level 3 = 100ns

80 of memory acess is found in level 1

705 " " " " " " level 2

$T_{avg} = 0.80 \times 5 + 0.20 \times 0.25 \times 25 + 0.20 \times 0.25 \times 125$

$= 14 ns$

DMA (Direct Memory Access)

	CPU involvement	Efficiency
programmed I/O	very high	low
Interrupt I/O	moderate	Better
DMA	very low	high

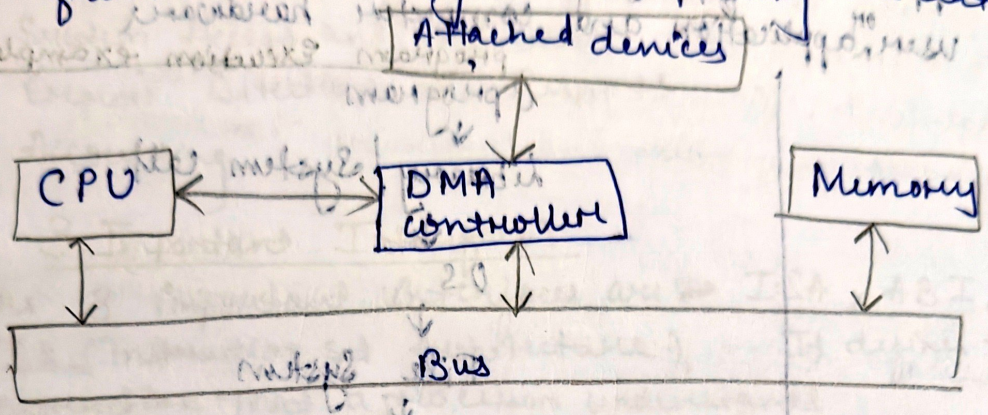
In programmed I/O the CPU itself keeps checking whether I/O device is ready. So this repeated checking is known as polling. In this case CPU efficiency is very low and CPU involvement is very high

Interrupt Driven I/O - Instead of continuously checking the device CPU tells the I/O device to inform when the execution is completed. CPU still has to participate in every data transfer. therefore CPU efficiency is better than the

programmed but not too much efficient.

DMA — DMA is used when large of data need to be transfer.

DMA allows an I/O device to transfer data directly to or from main memory without passing the data through CPU



Multiprocessor or Multicore

In single processor it executes instruction sequentically. But in multiprocessor the processor can execute simultaneously or parallely.

SMP (Symmetric Multiprocessor)

An SMP is computer system containing two or more similar processor that share the same main memory, I/O devices and are connected through a common interconnection system bus. These processors can perform the same function and controlled by a single integrated control system.
Symmetric Multiprocessor mean processor have equal capabilities

Multicore

A multicore computer contains two or more processor as core on single chips. It is also called chip multiprocessor.
In multicore computer each core has computer such as registers, ALU, pipeline hardware, control unit, L1 instruction cache memory, L1 data cache memory.

Ch-2 Operating System Overview

Operating System - is a system software that controls the execution of application / program and acts has an interface between the user or application and computer hardware

OS example

user
↓
Applications
↓
OS
↓
H/W

program Execution example

C program
↓
Library / System call
↓
OS
↓
File System
↓
Disk

Main Objective of OS

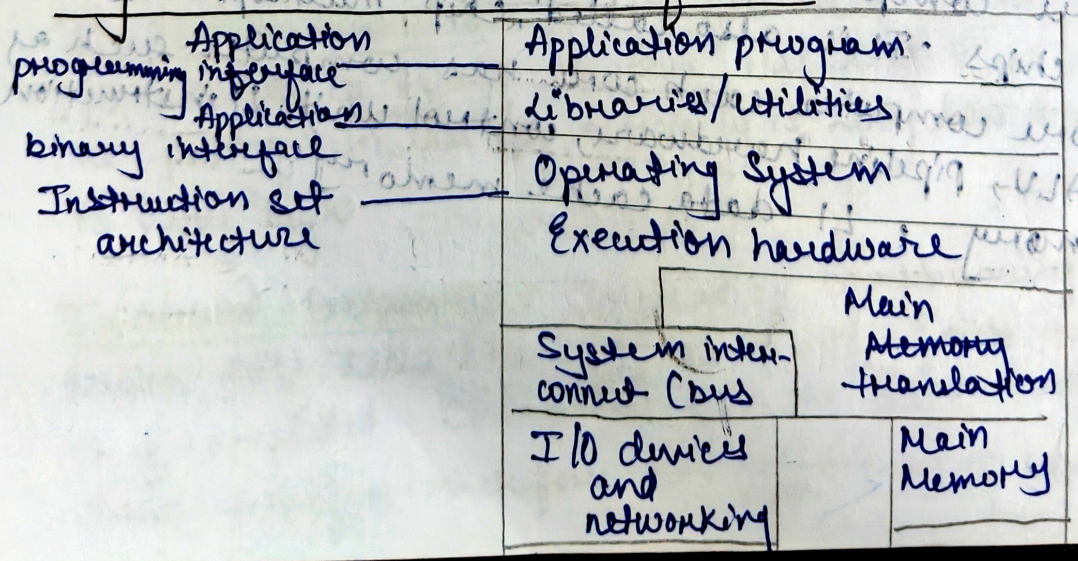
There are 3 major Objectives:-

(i) Convinence - The OS makes the computer easy to use

(ii) Efficiency - The OS ensure that hardware resources are used efficiently.

(iii) Ability to evolve: The OS should be designed so that the new features can be added without breaking the existing features. Ex - new hardware supports, new file system new security mechanism, new networking features, new system services

Major Function or services of a OS



There are 7 major function of OS

(i) Program Development
(ii) Program Execution
(iii) I/O device access
(iv) File access and file management
(v) System Access and Protection
(vi) Error Detection and Response
(vii) Accounting

3 Important Interface's

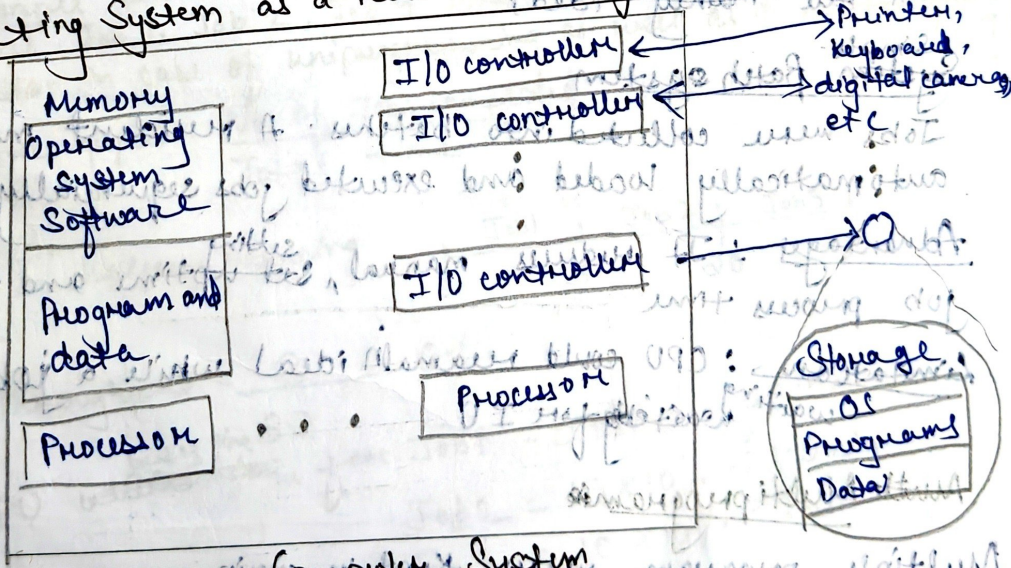
The 3 important interfaces are ISA, ABI, API.

ISA (Instruction set Architecture) :- It defines the machine instruction that a processor understands

ABI (Application binary interface): ABI defines a binary level interface between the application and system. It includes system call interface, binary format, compatibility and how programs interact with OS and hardware at binary level.

API (Applications Programming interface): API provides a programmer friendly interface to system services. Ex- Printf function, read function, write function.

Operating System as a Resource Manager



Computer System

The Operating System as Resource Manager

The Major Resources managed by OS are processor, main memory, Secondary Storage, I/O devices, files and data

Kernel

Kernel is a portion of OS that resides in main memory while the system is running. This Kernel performs essential like process management, file management, memory management, I/O management, protection and security hardware interaction.

Evolution of OS

Serial processing → Batch processing → Multiprogramming → Time Sharing

Main progress

Manual execution → Automation → Better CPU utilization → Interactive computing

In serial processing each program was manually loaded and executed one by one or one after another.

Advantage: This is simple and direct hardware control

Limitation: Large set up time, manual operation, and CPU remain idle mainly, ideal

Simple System Batch system

Jobs were collected into batches. A resident monitor automatically loaded and executed jobs sequentially

Advantages: It reduces manual setting time and improve job process time

Limitation: CPU could remain idle while a job is waiting for I/O

Multi programming

Multiple program were kept in main memory while one program waited for I/O The CPU switches to another program

Advantages: Better CPU utilization and higher throughput
Limitation: Required memory management, CPU scheduling protection and more complex OS design

Time Sharing System

* Execution process: Multiple users or programs share the CPU each received a small time quantum and the CPU repeated switched between them.

Advantages: Interactive computing, faster response for multiple user

Limitation: Context switching overhead, increase complexity in memory protection and resource management

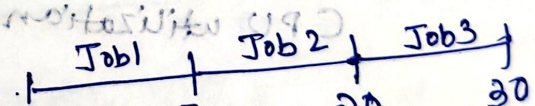
Q) Consider a computer with 400 MB of available memory. Suppose 3 programs Job 1, Job 2, Job 3 are submitted for purge programs and the data given.

Jobs	Total Duration	CPU usage	Memory
Job 1	5 min	70%	75 MB
Job 2	15 min	10%	100 MB
Job 3	10 min	10%	90 MB

For simple batch environment the jobs are executed in sequence Job 1, Job 2, Job 3. Find out the throughput and CPU utilization in case of uniprogramming as well as in multiprogramming

Throughput = $\frac{\text{No. of Jobs completed}}{\text{Total time}}$

For case1 uniprogramming



Throughput = $\frac{3}{30} = 0.1$

CPU utilization = $\frac{\text{busy time}}{\text{total time}}$

for Job 1 = 5 × 70% = 3.5 min
Job 2 = 15 × 10% = 1.5 min
Job 3 = 10 × 10% = 1 min

Total CPU busy time = 3.5 + 1.5 + 1 = 6 min

CPU utilization for uniprogramming = $\frac{6}{30} \times 100 = 20\%$

Multiprogramming

throughput = $\frac{3}{15} = 0.2$ min

total time = 15 min

CPU utilization $\Rightarrow \frac{6}{15} \times 100$

Note : All Job can do I/O operation in same time but cannot do CPU operation at same time

	CPU and I/O pattern
J_1	CPU 3ms → I/O 4ms → CPU 5 MS
J_2	CPU 2ms → I/O 5ms → CPU 3 ms
J_3	CPU 4ms → I/O 3ms → CPU 1 ms

Q Find out the CPU utilization and throughput for uniprogramming and multiprogramming both

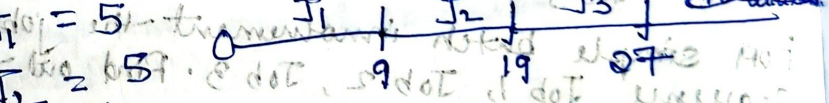
Uniprogramming

CPU busy time $J_1 = 5$

$J_2 = 5$

$J_3 = 5$

15



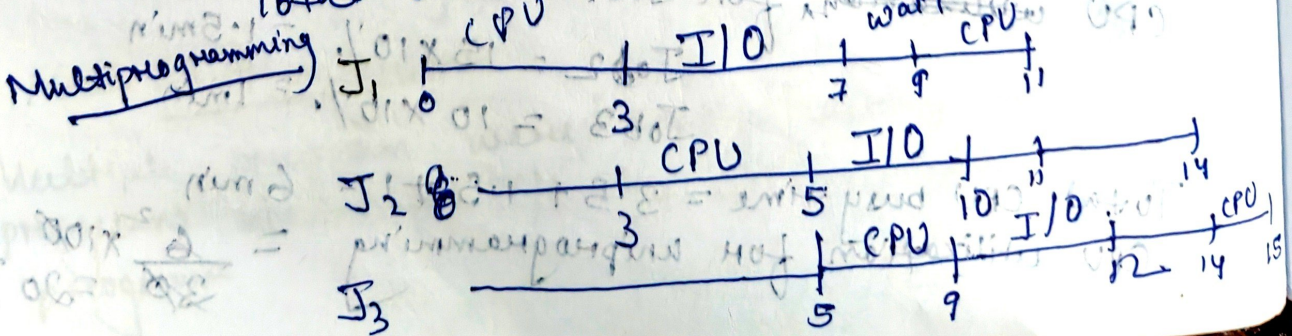
Total execution time = 27

CPU utilization $= \frac{15}{27} \times 100 = 55.56$

Throughput $= \frac{3}{27} = 0.111$

Multiprogramming

Total execution time = 27



Total execution time = 15

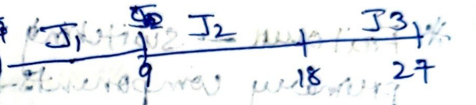
CPU utilization $\frac{15}{15} \times 100 = 100\%$

Throughput $= \frac{3}{15} = 0.20$

Q)

J_1	I/O 3ms	CPU 4ms	I/O 2ms
J_2	I/O 2ms	CPU 3ms	I/O 4ms
J_3	I/O 4ms	CPU 2ms	I/O 3ms

Uniprogramming



CPU busy time $J_1 = 4$

$J_2 = 3$

$J_3 = 2$

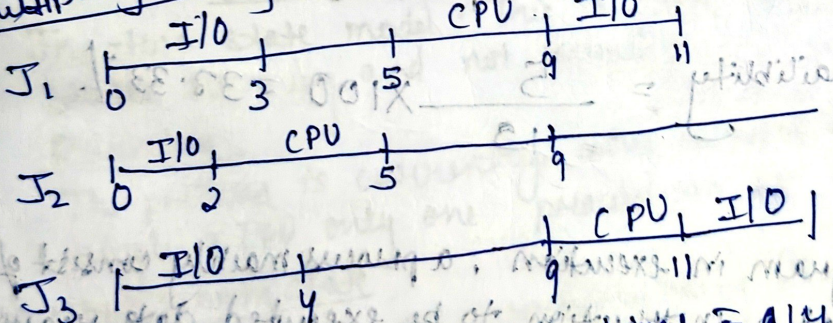
9

Total execution time = 27

CPU utilization $= \frac{9}{27} \times 100 = 33.33\%$

Throughput $= \frac{3}{27} = 0.11\%$

Multiprogramming



Total execution time = 14

CPU busy time = 9

CPU utilization $= \frac{9}{14} \times 100 = 64.28\%$

Throughput $= \frac{3}{14}$

Fault Tolerance

* It is the ability of an OS to continue functioning even if some hardware or software component fail.

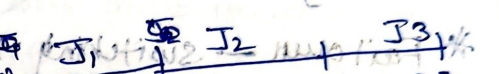
Total execution time = 15

CPU utilization $\frac{15}{15} \times 100 = 100\ ns$

Throughput $= \frac{3}{15} = 0.20$

b)

J_1	I/O 3ms →	CPU 4ms →	I/O 2ms
J_2	I/O 2ms →	CPU 3ms →	I/O 4ms
J_3	I/O 4ms →	CPU 2ms →	I/O 3ms



Uniprogramming

CPU busy time $J_1 = 4$

$J_2 = 3$

$J_3 = 2$

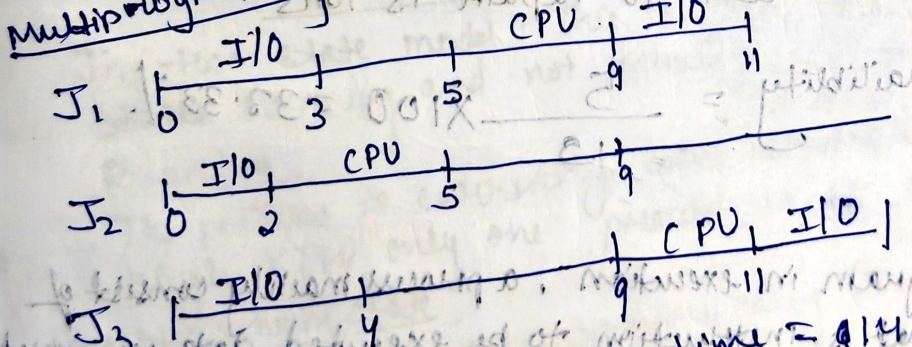
9

Total execution time = 27

CPU utilization $= \frac{9}{27} \times 100 = 33.33\%$

Throughput $= \frac{3}{27} = 0.11\ \%$

Multiprogramming



Total execution time = 14

Throughput $= \frac{3}{14}$

CPU busy time = 9

CPU utilization $= \frac{9}{14} \times 100 = 64.28\ \%$

Fault Tolerance

It is the ability of an OS to continue functioning even if some hardware or software component fail

* It main role is to improve reliability, availability of the system
* The main technique of fault tolarence include redundancy components. (keeping backup hardware or software), maintaining copy of data
* Disk monitoring * Check point - saving the system state so execution can resume after failure
* Error detection and recovery - detecting errors and taking corrective actions
* Failover - switching to a back up components when the primary components fail

To find the availibility where MTTF represents mean time to failure and MTTR represents mean time to repair

$$\text{Availibility} = \frac{MTTF}{MTTF + MTTR} \times 100$$

Q Calculate the availibity of a system if its mean time to failure is 5hrs mean to repair is 10hrs

Ans $$\text{Availiblity} = \frac{5}{15} \times 100 = 33.33\%$$

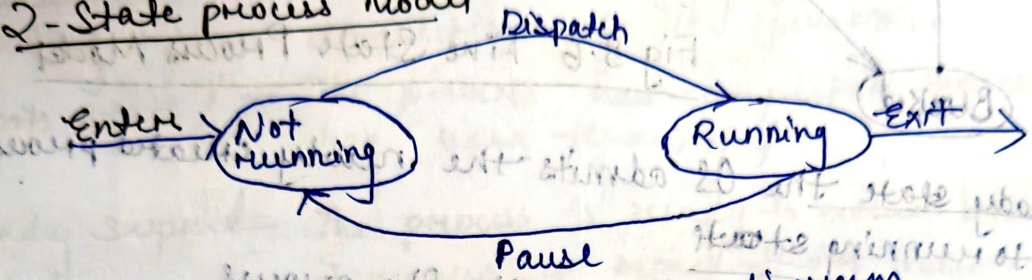
Process

A process in a program in execution. a process mainly consist of program code that is instruction to be executed data required by the program. Process Control Block (PCB) information maintained by the OS about the process. PCB is a data structure maintained by OS that stores all imp information about a process

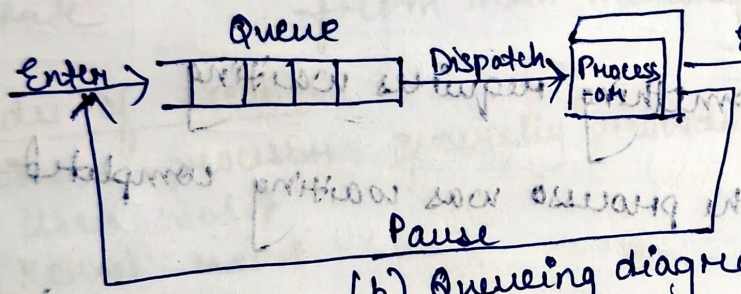
(i) Process ID
(ii) " State

(iii) Priority
(iv) Program Counter
(v) context data
(vi) & Memory information
(vii) I/O status information
viii) Accounting information

2-State process Model



(a) State transition diagram



(b) Queueing diagram

TWO - State Process Model

In two state model. This is the simplest model having two states running and not running

Running
The process is currently being executed by the CPU. and for a single a CPU only one process can be running at a time.

Not running state
The process is not currently using the CPU. It is waiting for its turn to execute

Limitation
Ex Consider two processon are their, P_1 process waiting for CPU which is in ready queue and P_2 process waiting for I/O which is in blocked state. Both processes are not running but their situation are completely different.

5 State process Model

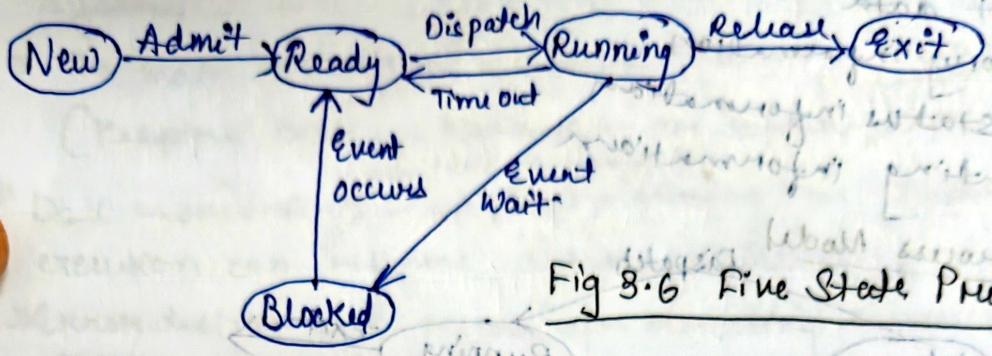


Fig 3.6 Five State Process Model

New to ready state the OS admits the newly created process

Ready to running state

The scheduler or dispatcher selects the process

Running to ready state

This is usually because time out or preemption

Running to block state

The process request something requires waiting

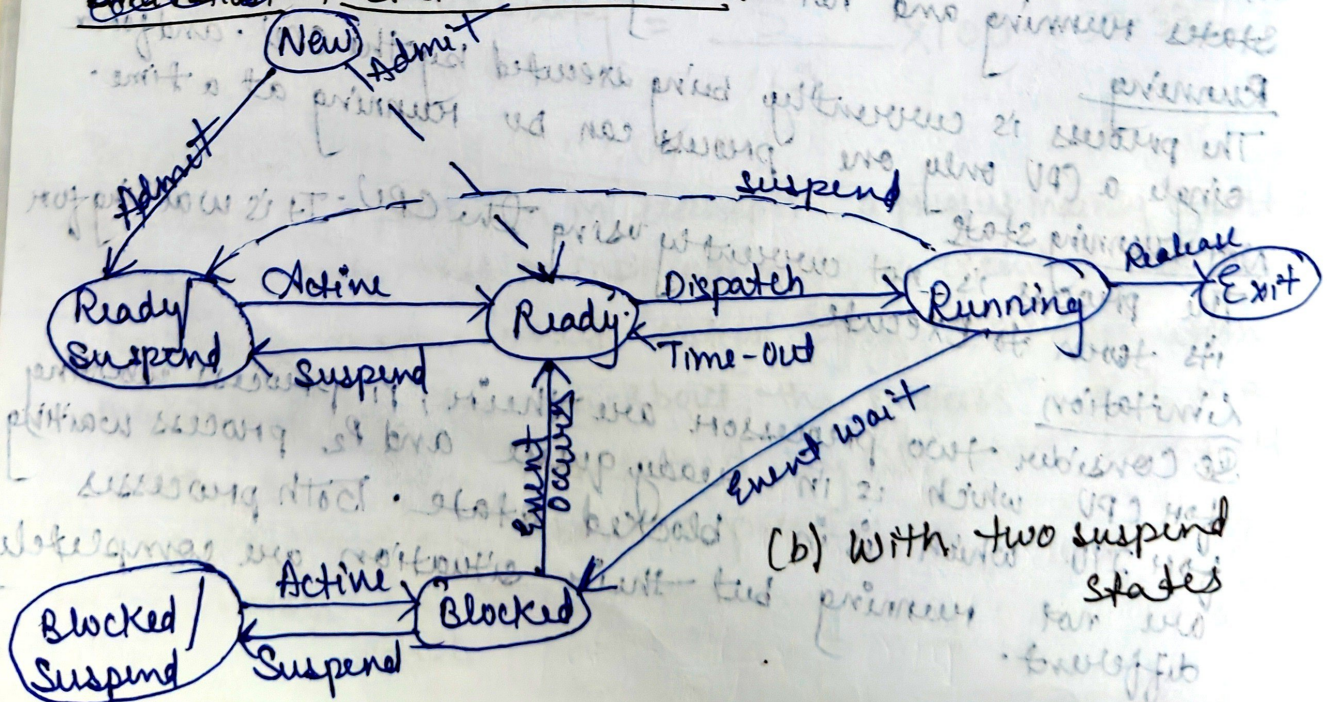
Block to ready state

The event for which the process was waiting completed

Running to exit

The process may completes normally or it is terminated because of one error

7 state Process Model



(b) With two suspend states

New state – The process has been created but has not yet been admitted for execution

Ready state – The process is in main memory and ready to execute but waiting for the CPU.

Running state – The process is currently being executed by the CPU

Blocked state / waiting state – The process is waiting for an event such as completion of I/O operation.

Exits state / – The process has completed ~~exe~~ execution or has been terminated

Ready suspend state – The process is ready to execute but has been tempararily moved from main memory to secondary storage

Blocked suspend state – The process is waiting for an event and moved from main memory to secondary storage

Modes of execution

Modern processor generally provide 2 execution mode

(i) User mode
(ii) Kernel mode

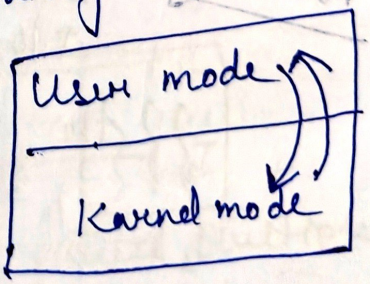
User mode

Used by normal application or program and it has limited privileges

Kernel mode

Used by the OS. It has higher privileges and can executed privileged instruction

User program
↓
System call
↓
Kernel mode
↓
OS performs the service
↓
User mode

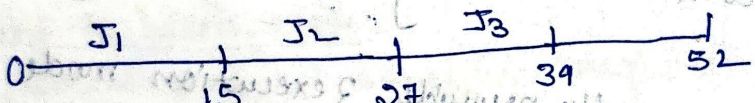


Q) Four Jobs J1, J2, J3, J4 are submitted to a computer system at time t = 0. Each Job performs an I/O operation, followed by a CPU operation and another I/O operation. The required time for Job1 is 4ns, 6ns, 5ns on J2 3ns, 5ns, 4ns for J3 5ns, 4ns, 3ns and for J4 2ns 7ns 4ns. Find the CPU utilization for uniprogramming and multiprogramming both

	I/O	CPU	I/O
J_1	4	6	5
J_2	3	5	4
J_3	5	4	3
J_4	2	7	4

Ans CPU busy time = 6 + 5 + 4 = 22

This type of question will definitly come

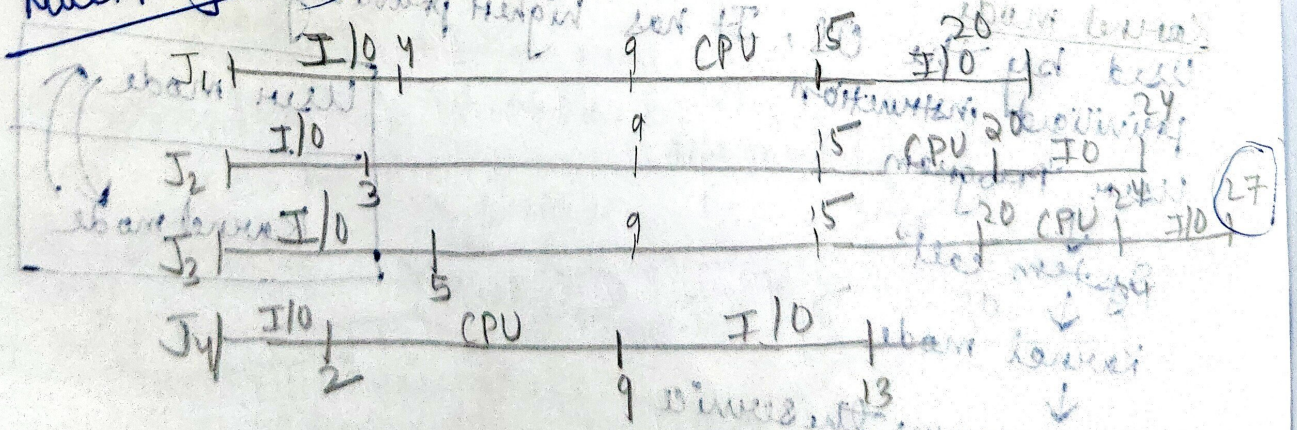


Total execution time = 52

CPU utilization = $\frac{22}{52} \times 100 = 42.30$

Throughput = $\frac{4}{52} = 0.07$

Multiprogramming



Total execution = 27

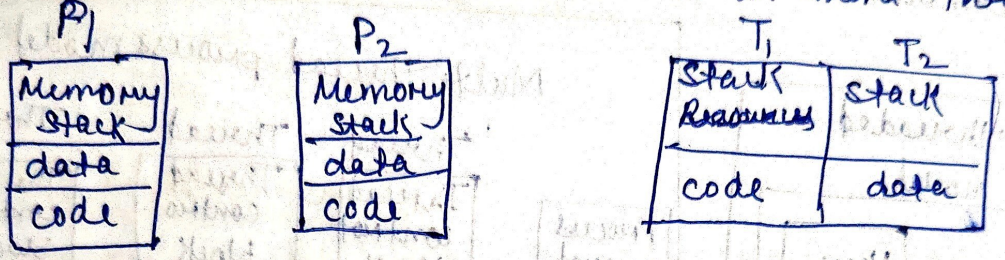
CPU utilization = $\frac{22}{27} \times 100 = 81.48$

Throughput = $\frac{4}{27} = 0.14$

Thread in Operating System Ch-4

A process is a program in execution and it is mainly considered a unit of resource ownership and protection.

A thread is a unit of execution on CPU scheduling within a process. A process can contain one or more threads.



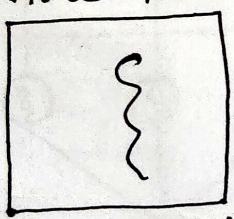
The Threads share process resources but each thread has its own execution and stack.

What is shared in thread?

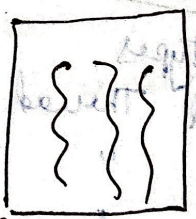
The thread share code, data, heap, files, Address space other process resources.

Each thread has its own program counter, CPU registers, Stack thread state, Thread control block.

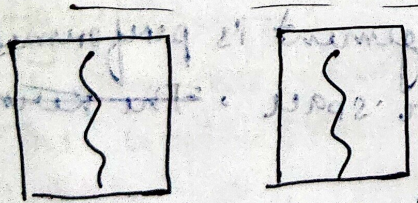
~~Single threads~~



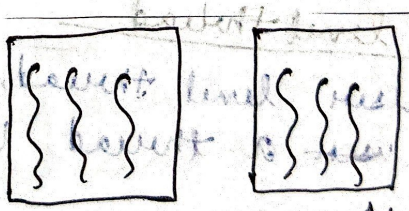
One process One thread



One process multiple threads



Multiple process One thread per process



Multiple process Multiple threads per process

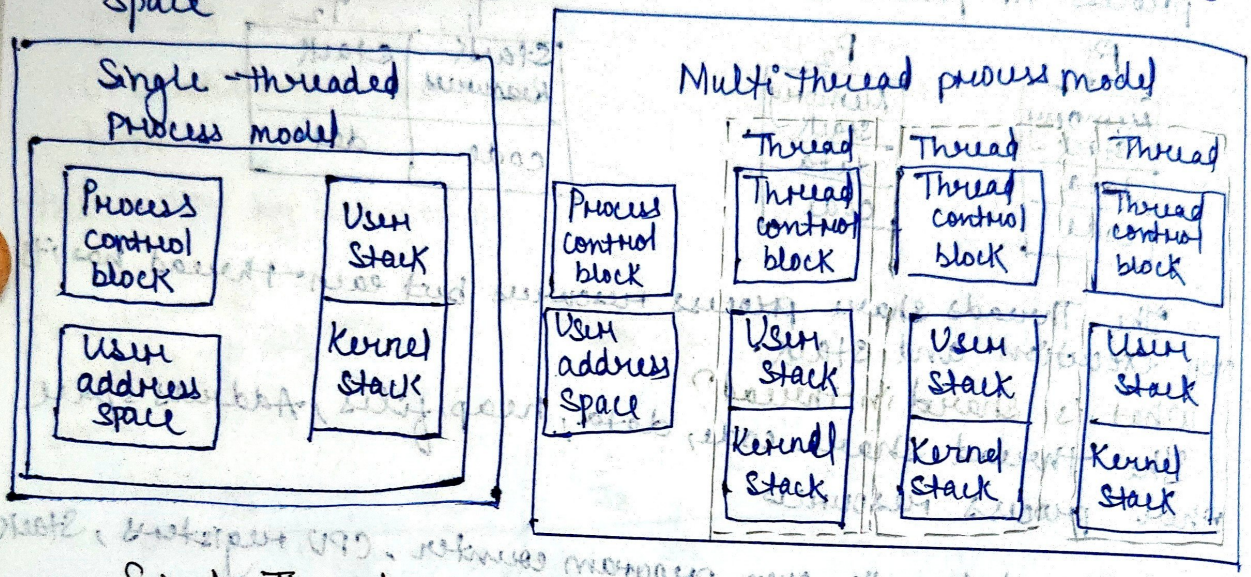
Needs of threads

i) faster creation - creating a thread is generally much cheaper than creating a new process.

ii) Faster termination - a thread requires fewer resources to terminate than a process

iii) Faster context switching - Switching between the threads of the same process is generally is cheaper than switching between separate process

iv) Easy communication - Threads share the same address space



Single Thread and Multithread Process Model

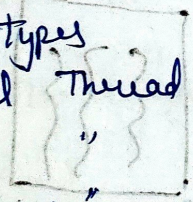
Types of Thread

Mainly 3 types

(i) User level Thread

(ii) Kernel " "

(iii) Hybrid/combined "





User level thread

In User level thread, thread management is performed by a thread library in user space. The Kernel does not directly know about individual state

Advantages

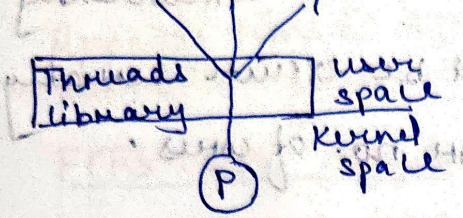
Thread creation is faster

Thread switching " "

No Kernel involvement is required for many thread operation

Limitation

* If one user-level-thread makes a blocking-system for the entire process, the entire process may become block because the Kernel consider only the process



Pure-User-level

Kernel Level Thread

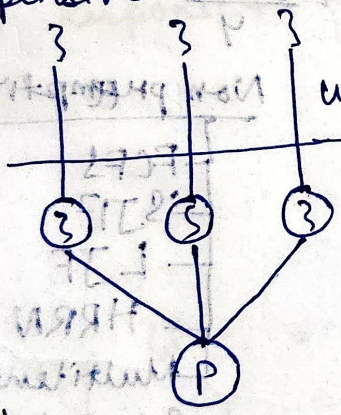
In Kernel level thread the OS permit known about the threads and manages the threads

Advantages

If one thread blocks for I/O operation the other threads can continue execution

Disadvantages/Limitation

Kernel involvement makes thread management more expensive the user level thread management

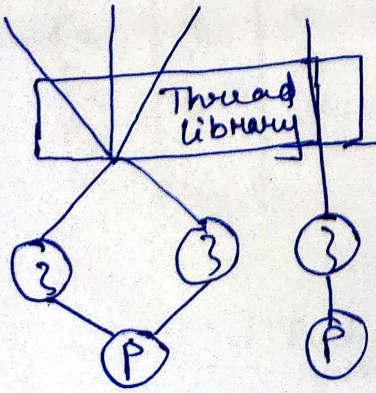


(b) Pure - Kernel-level

Hybrid =

It combines user level thread and Kernel level thread. The application may have several user level threads which are mapped into Kernel level threads.

* No Limitation



(C) Combined

Amdhal's law

Amdhal's law states that speed up $= \frac{1}{1-f+\frac{f}{N}}$

where f is fraction of program that can be concurrently execute.

1-f represents fraction that must be execute serially and N represents no. of processors or no. of cores.

Q Suppose 90% of the program is parallizable and 10% is serial and 8 core are available the what is the speed up?

Ans Speed up $= \frac{1}{(1-0.9)+\frac{0.9}{8}} = \frac{0.1}{}$

Q) f = 75%. available core = 4

Ans speed up $= \frac{1}{(1-0.75)+\frac{0.75}{4}}$

CPU Scheduling

Pre-emptive

- SRTF
- LRTF
- Round Robin
- priority based scheduling

Non preemptive

- FCFS
- SJF
- LJF
- HRRN
- Multilevel Queue Scheduling

1. Arrival Time
2. Burst Time
3. Completion Time
4. Turn around time
5. waiting time
6. Response Time

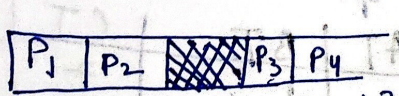
* Completion time is equal to the time at which the process completes its execution.
* Turn around time = completion Time - Arrival time
* Waiting time = Turn around Time - Burst time
* Response time is the time at which a process get CPU first time - arrival time.

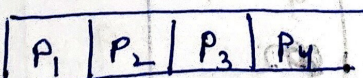
FCFS CPU Scheduling

Processor	AT	BT	CT	TAT	WAT	RT
P_1	0	2	2	2	0	0
P_2	1	2	4	3	1	1
P_3	5	3	8	3	0	0
P_4	6	4	12	6	2	2

Arrival
Burst time

Find the completion time, turn around time, waiting time, Response time





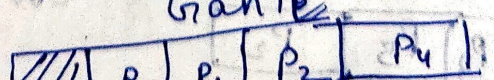
Average TAT = 3.5
Average WT = 0.75

SJF (Shortest Job First)

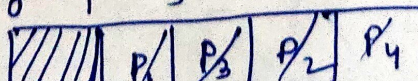
Process	AT	BT	CT	TAT	WAT	RT
P_1	1	3	6	5	2	2
P_2	2	4	10	8	4	4
P_3	1	2	3	2	0	0
P_4	4	4	14	10	6	6

Average TAT =

Gantt chart



Ready queue



SRTF (Shortest Remaining Time First)

process Id	AT	BT	CT	TAT	WT
P_1	0	5 4	9	9	5
P_2	1	3 ~~2 1 0~~	4	3	0
P_3	2	4	13	11	7
P_4	4	~~1 0~~	5	1	0

Ready queue

P_1	~~P_2~~	P_3	~~P_4~~

0

Gantt chart

P_1	P_2	P_2	P_2	P_4	P_1	P_3

0 1 2 3 4 5 9 13

average TAT = 6

average WT $\frac{5+7}{4} = 3$

SJF

SRTF

SJF →

process id	AT	BT	CT	TAT	WAT	RT
P_1	3	1	7	4	3	
P_2	1	4	16	15	11	
P_3	4	2	9	5	3	
P_4	0	6	6	6	0	
P_5	2	3	12	10	7	

SJF

Ready queue

P_4	P_1	P_3	P_5	P_2

0

Gantt chart

P_4	P_1	P_3	P_5	P_2

0 6 7 9 12 16

Total execution time = 16

average TAT = 10

" WT = 6

SRTF

RQ | P_4 | P_2 | P_5 | P_1 | P_3 |
0

GC | P_4 | P_2 | P_2 | P_1 | P_2 | P_2 | P_3 | P_5 | P_4 |
0 1 2 3 4 5 6 8 11 15

* No matter if you are taking SJF or SRTF execution time is same

Process Id	AT	BT	CT	TAT	WAT	RT
P_1	3	1	4	1	0	
P_2	1	4	6	5	1	
P_3	4	2	8	4	2	
P_4	0	6	15	15	9	
P_5	2	3	11	9	6	

RR (Round Robin CPU Scheduling)

TQ = 2unit

Pm	AT	BT
P_1	0	~~5~~ 3 1
P_2	1	~~4~~ 2 0
P_3	2	~~2~~ 0
P_4	4	1 0

Ready queue

P_1	P_2	P_3	P_1	P_4	P_2	P_1

GT | P_1 | P_2 | P_3 | P_1 | P_4 | P_2 | P_1 |
0 2 4 6 8 9 10 11

Q)

P	AT	BT
P_1	0	5
P_2	1	3
P_3	2	1
P_4	3	2
P_5	4	3

QT = 2 units

RQ | P_1 | P_2 | P_3 | P_1 | P_4 | P_5 | P_2 | P_1 | P_5 |

GT | P_1 | P_2 | P_3 | P_1 | P_4 | P_5 | P_2 | P_1 | P_5 |

0 2 4 5 7 9 11 12 13 14

Process	AT	BT
P_1	5	5
P_2	4	6
P_3	3	7
P_4	1	9
P_5	2	2
P_6	6	3

Q = 3 units

	P_4	P_5	P_3	P_2	P_1	P_6	P_4	P_3	P_2	P_1	P_4	P_3

0 1 4 7 10 13 16 19 22 25 28 31 34

Priority CPU scheduling

Process	AT	BT	Priority
P_1	0	5	10
P_2	1	4	20
P_3	2	2	30
P_4	4	1	40

Higher the value Higher the priority

RQ | P1 | P2 | P3 | P4

GT | P1 | P2 | P3 | P4 | P2 | P1 |
0 1 2 4 5 8

P	AT	BT	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

| P1 | P2 | P3 | P4 | P5 |

| P1 | P2 | P3 | P4 | P5 |
0 1 2 3 8

Multilevel Queue Scheduling

Imp

Highest priority

- System processes
- Interactive process
- Batch process

Lowest priority

This scheduling algorithm is created for area in which we classify process into different group. Such as lowest priority process and highest priority process.

The multilevel queue scheduling algorithm partition the readyqueue into several separate.

Each queue is associated with its own scheduling algorithm. ~~For the~~ No process in batch queue dragon ~~under~~ unless for the system queue and interactive process were empty.

Q

process	BT	Queue No	AT
P_1	5	1	0
P_2	3	1	0
P_3	8	2	0
P_4	2	1	10

Priority of ~~queue~~ Q1 is greater than priority of Q2 and Q1 uses Round Robin scheduling with quantum time 2. Q2 uses FCFS

Ans

Ready Queue

P_1	P_2	P_3	P_1	P_2	P_1

P_1	P_2	P_1	P_2	P_1	P_3	P_4	P_3

0 2 4 6 7 8 10 12 18

Q

process	BT	AT	Queue
P_1	10	0	2
P_2	7	3	1
P_3	6	4	2
P_4	5	12	1
P_5	8	18	1

Queue 1 > Queue 2

RR (QT = 4) PR (QT = 3)

Ans

Ready Queue

Q1 | P_2 | P_4 | P_4 | P_5 |

Q2 | P_1 | P_3 | P_1 | P_3 |

RQ

GC | P_1 | P_2 | P_2 | P_3 | P_4 | P_4 | P_5 | P_1 | P_3 | P_1 |

0 3 7 10 13 17 18 26 29 32 36

RQ

P_1	P_2	P_1	P_3	P_2	P_1	P_4	P_5	P_3	P_1

GC

P_1	P_2	P_2	P_1	P_4	P_4	P_5	P_3	P_1	P_3	P_1
0	3	7	10	13	17	18	26	29	32	35	36

Q) Q_1 — RR Q_2 — SRTF

process	AT	BT	Queue
P_1	0	5	Q_1
P_2	1	4	2
P_3	2	3	1
P_4	4	6	2
P_5	5	2	1
P_6	6	4	2

@ Q_1 ($QT_1 = 2$)

Multilevel Queue Scheduling

Queue	Priority	Scheduling algo	Processes	AT	BT
Q_1	1	RR (QT = 2)	P_1	7	2
Q_2	2	SRTN	P_2	4	6
			P_3	4	4
Q_3	3	FCFS	P_4	2	3
			P_5	0	13

$Q_1 > Q_2 > Q_3$

Ans

RQ | P_5 | P_4 | P_2 | P_3 | P_1 |

P_5	P_5	P_5	P_5	P_3	P_3	P_3	P_1	P_3	P_2	P_5	P_4
0	2	2	3	4	5	6	7	9	10	16	24	27

Q)

P	BT	AT	Queue
P_1	10	0	2
P_2	7	3	1
P_3	64	4	2
P_4	53	12	1
P_5	8	18	1

$Q_1 > Q_2 \rightarrow$ SJF

RR (QT=3)

RQ | P_1 | P_2 | P_3 | P_4 |

GC	P_1	P_2	P_2	P_3	P_4	P_4	P_3	P_5	P_3	P_1
0	3	6	10	12	15	17	18	26	29	36

Highest Response Ratio Next (HRRN) Scheduling

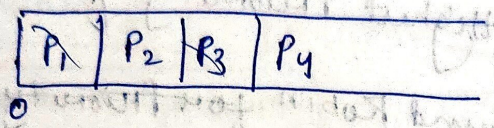
HRRN is a non ~~primitive~~ preemptive scheduling which choose the next process to execute based on the parameter ratio

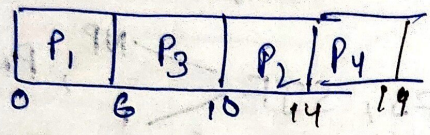
$$\text{Response Ratio} = \frac{(w+s)}{s}$$

W is the waiting time of the process upto till now
S is the burst time of the process

Ex

	AT	BT
P_1	0	6
P_2	4	10
P_3	4	4
P_4	8	5





Waiting time = 6 − 4 = 2

P_2 = RR = $\frac{W+S}{2} = \frac{2+10}{10} = 1.2$, RR($P_3$) = $\frac{2+4}{4} = 1.5$

Waiting time = 10 − 4 = 6

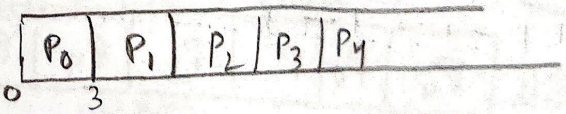
P_2 → RR = $\frac{6+10}{10} = 1.6$

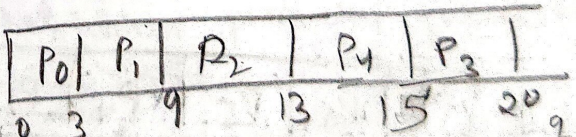
Waiting time = 10 − 8 = 2

P_4 RR = $\frac{2+5}{5} = \frac{7}{5} = 1.4$

Q)

	AT	BT
P_0	0	3
P_1	2	6
P_2	4	4
P_3	6	5
P_4	8	2





9/4

① P_2 = 9 − 4 = 5 RR = $\frac{5+4}{4} = 2.25$

P_3 = 9 − 6 = 3 RR = $\frac{3+5}{5} = 1.6$

P_4 = 9 − 8 = 1 RR = $\frac{1+2}{2} = \frac{3}{2} = 1.5$

② P_3 = 13 − 6 = 7 RR = $\frac{7+5}{5} = \frac{12}{5} = 2.4$

P_4 = 13 − 8 = 5 RR = $\frac{5+2}{2} = \frac{7}{2} = 3.5$

Substitute

Process	Priority	AT	BT
P_1	5	0	14
P_2	4	1	12
P_3	3	3	10
P_4	2	4	8
P_5	2	4	6
P_6	1	6	2

Lowest Priority Number = Highest Priority for Priority Scheduling

Time Quantum = 3 for Round Robin ~~for Priority Scheduling~~

~~Time Quantum = 3~~

Prepare Gantt Chart for FCFS, SJF (NP, P), Priority (NP, P), Round Robin and HRRN

Any FCFS

RQ | P_1 | P_2 | P_3 | P_4 | P_5 | P_6 |

0 14

GC | P_1 | P_2 | P_3 | P_4 | P_5 | P_6 |

0 14 26 36 44 50 52

SJF

RQ | P_0 | P_1 | P_2 | P_3 | P_4 | P_5 | P_6 |

GC | P_0 | P_6 | P_5 | P_4 | P_3 | P_2 |

0 14 16 22 30 40 52

③ SRTF RQ | P_1 | P_2 | P_3 | P_4 | P_5 | P_6 |

GC | P_1 | P_2 | P_2 | P_2 | P_5 | P_5 | P_6 | P_5 | P_4 | P_2 | P_3 | P_1 |

0 1 2 3 4 5 6 8 12 20 29 39 52

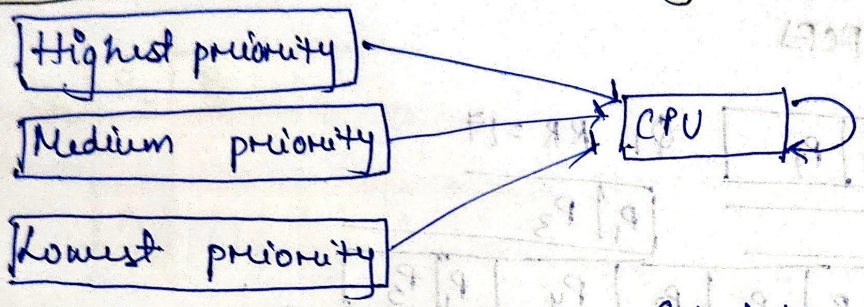
Priority

P_1	P_2	P_3	P_4	P_5	P_6

P_1	P_6

0 14 16

Multilevel Feedback Queue Scheduling.



* Multilevel Feedback Queue Scheduling is a CPU scheduling algorithm that maintains multiple ready queue. Each queue with different priority level and time quantum values.
* New process start at the highest priority queue and based on the behaviour they may be promoted demoted between the queues.
* This adaptive approach balances the needs of both interactive and CPU intensive process. This multilevel feedback Queue scheduling, where high priority based scheduling where higher priority are served first.
* Variable time quantum that means higher priority queues have shorter time slice
* Dynamic priority adjustment means process moves between queues based on behaviour
* Aging mechanism - it prevents starvation by promoting long waiting process

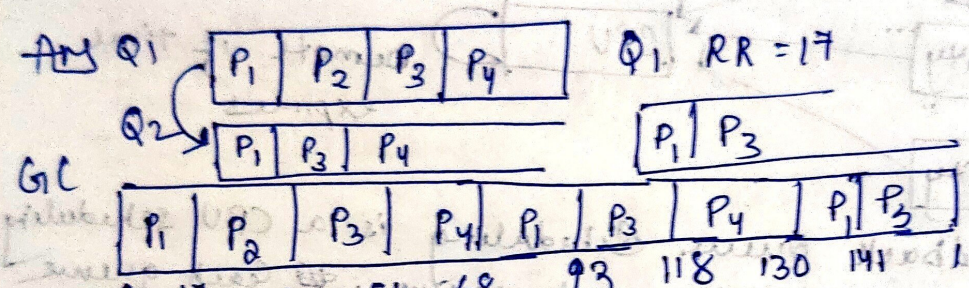
Q)

Process	BT	AT
P_1	53	0
P_2	17	0
P_3	68	0
P_4	24	0

First queue use RR with $q_t = 17$

Q_2 " RR " " 25

Q_3 " FCFS



Q1 RR = 17

Q)

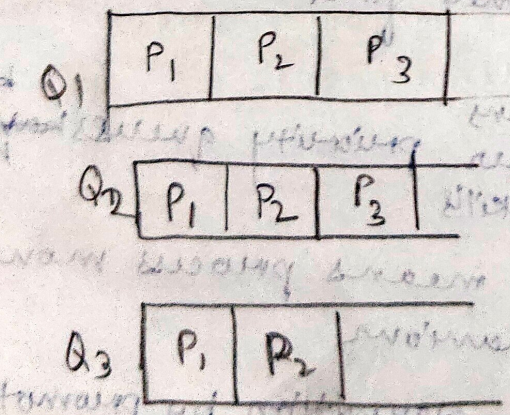
Process	AT	BT
P_1	0	8 7 5
P_2	1	4 3 1
P_3	2	2 1

$Q_1 > Q_2 > Q_3$

Q_1 = RR (QT = 1)

Q_2 = RR (QT = 2)

Q_3 = FCFS



Q

Process	AT	BT	Q No
P_1	0	5	1
P_2	0	31	2
P_3	0	8	2
P_4	0	6	1

Q_1 = FCFS $Q1 > Q_2$

Q_2 = RR QT = 2

As it is in highest priority so not need to do 1 unit

Q_1 | P_1 | P_4 |

Q_2 | P_2 | P_3 |

P_1	P_4	P_2	P_3	P_2	P_3

0 5 11 13 15 16 22

Q)

Process	AT	BT	Priority
P_1	0	5	1
P_2	1	3	2
P_3	2	8	1
P_4	3	6	3

Highest value = Highest priority

(Preemptive priority scheduling)

means if higher priority queue comes then previous queue has to wait

RQ | P_1 | P_2 | P_3 | P_4 |

P_1	P_2	P_4	P_2	P_1	P_3

0 1 3 9 10 14 22

Chapter-5

Process Synchronization

Process Synchronization

It is a mechanism used in a operating system to coordinate the execution of multiple process or threads that share common resources or data.

Its main purpose is to ensure that only one process access a shared resource at a time. To prevent in consistance result.

Some common synchronization mechanism include mutual exclusion, Semaphores, Monitors, Message Passing, Readers/Writers Problem

Mutual Exclusion

Mutual Exclusion or Mutex is a synchronization principle that ensure only one process or thread can enter the critical section at a time.

If one process is using the shared resource then other process must wait untill the first process finishes the critical section

Dekker's Algorithm

The Dekker's Algorithm is software based solution for achieving mutual exclusion bet 2 process

```
/* PROCESS 0 */
:
while (turn != 0)
    /* do nothing */;
/* critical section */;
turn = 1
```

```
/* PROCESS 1 */
:
while (turn != 1)
    /* do nothing */;
/* critical section */;
turn = 0;
```

(a) First Attempt

For first attempt the variable turn ~~tell the proc~~ tell which process has access to enter the critical section.
If turn value is 0 process P0 then enter the critical section
If turn value is 1 process 1 can access the critical section.

Limitation in first attempt in Dekker's Algorithm

1) Strict alternation - process must alternate even when other process does not need the critical section given
2) Permanent Blocking can be the cause of failure — If one process fails the other may wait forever

2nd Attempt

```
/* PROCESS 0 */
:
while (flag[1])
  /* do nothing */;
flag[0] = true;
/* critical section */;
flag[0] = false;
```

```
/* PROCESS 1 */
:
while (flag[0])
  /* do nothing */;
flag[1] = true;
/* critical section */;
flag[1] = false;
```

boolean flag[2] = [false, false]
P0 — P1

The second attempt of Dekker's Algorithm uses boolean flag array to maintain the status of both process. Flag[0] represent process P0 and ~~flag~~ Flag[1] represent process P1. False means process is not trying to enter the critical section and true means process is " " " " section.

Advantages of ~~Second~~ Second attempt

1) Each process has its own flag
2) If one process fails other process is not blocked

Limitations

1. The algorithm does not guarantee mutual exclusion. When both process can simultaneously check the other process's

flag as false the both process set own flag to true and enter the critical section